

Tree Algorithms

CMPT 145

Copyright Notice

©2020 Michael C. Horsch

This document is provided as is for students currently registered in CMPT 145.

All rights reserved. This document shall not be posted to any website for any purpose without the express consent of the author.

Learning Objectives

After studying this chapter, a student should be able to:

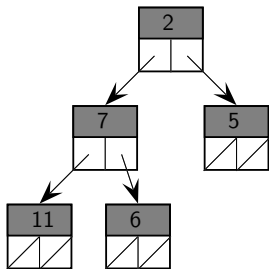
- Describe pre-order, in-order, and post-order traversals, and demonstrate their behaviour in example binary trees.
- Employ a binary tree as a solution to a software design problem.

Sequences and Tree Traversals

- Sequential data stored in a list has a unique ordering.
- Data stored in a tree does not have a unique sequence.
- There are 4 distinct sequences that an algorithm could use to explore a tree.
- Each sequence can be expressed as an algorithm, called a traversal.
- Almost every algorithm you need to work with data stored in a tree will be based on one of these traversals.

Breadth Sequence

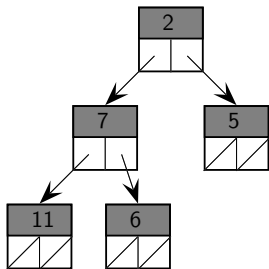
1. Each level from top to bottom, from left to right
2. Siblings before children



Breadth sequence: 2 7 5 11 6

Breadth Sequence Formally

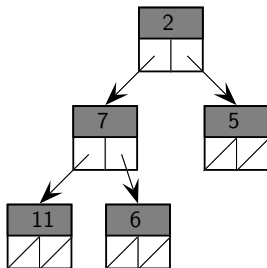
1. Nodes at level i before nodes at level $i + 1$
2. From left to right



Breadth sequence: 2 7 5 11 6

Pre-Order Sequence

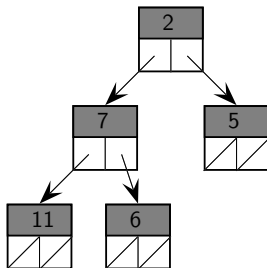
1. Root before children
2. Children from left to right



Pre-Order sequence: 2 7 11 6 5

Post-Order Sequence

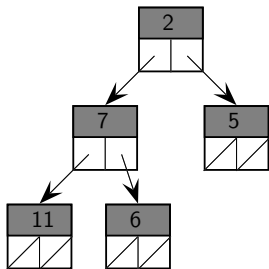
1. Children before root
2. Children from left to right



Post-Order sequence: 11 6 7 5 2

In-Order Sequence

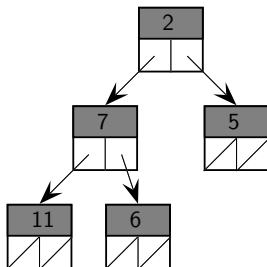
1. Left child before root
2. Root before right child



In-Order sequence: 11 7 6 2 5

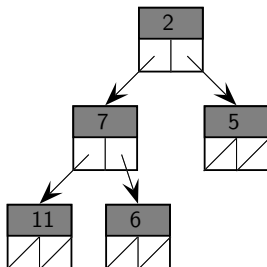
Pre-Order Sequence by hand

1. Write down the root.
2. Draw a box for left and right subtrees **after** the root.
3. Fill in each box using Pre-order Sequence.



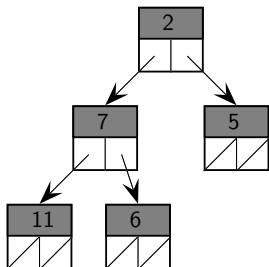
Post-Order Sequence by hand

1. Draw a box for left and right subtrees.
2. Write down the root **after** the boxes
3. Fill in each box using Post-order Sequence.



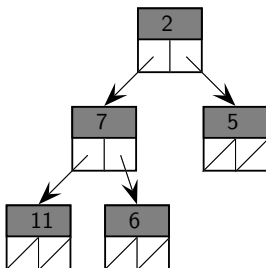
In-Order Sequence by hand

1. Draw a box for left and right subtrees.
2. Write down the root **between** the two boxes.
3. Fill in each box using In-Order Sequence.



Pre-Order Traversal: Pseudo-code

- If the tree is empty, do nothing.
- Otherwise:
 1. Process the root of the subtree.
 2. Recursively process the left-subtree in pre-order sequence
 3. Recursively process the right-subtree in pre-order sequence



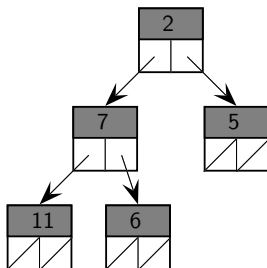
Pre-Order Traversal algorithm

```
1 def pre_order(tnode):  
2     if tnode is None:  
3         return  
4     else:  
5         print(tnode.get_data(), end=" ")  
6         pre_order(tnode.get_left())  
7         pre_order(tnode.get_right())
```

Root first; left recursively, then right.

In-Order Traversal: Pseudo-code

- If the tree is empty, do nothing.
- Otherwise:
 1. Recursively process the left-subtree in in-order sequence
 2. Process the root of the subtree.
 3. Recursively process the right-subtree in in-order sequence



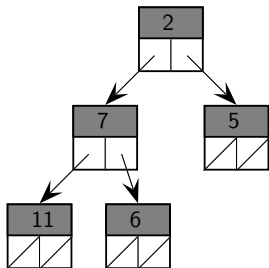
In-Order Traversal algorithm

```
1 def in_order(tnode):  
2     if tnode is None:  
3         return  
4     else:  
5         in_order(tnode.get_left())  
6         print(tnode.get_data(), end=" ")  
7         in_order(tnode.get_right())
```

Left recursively first; root, then right.

Post-Order Traversal: Pseudo-code

- If the tree is empty, do nothing.
- Otherwise:
 1. Recursively process the left-subtree in post-order sequence
 2. Recursively process the right-subtree in post-order sequence
 3. Process the root of the subtree.



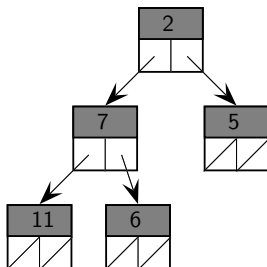
Post-Order Traversal algorithm

```
1 def post_order(tnode):  
2     if tnode is None:  
3         return  
4     else:  
5         post_order(tnode.get_left())  
6         post_order(tnode.get_right())  
7         print(tnode.get_data(), end=" ")
```

Left recursively first; right, then root.

Breadth Traversal: Pseudo-code

1. Each level from top to bottom, from left to right
2. Siblings before children



Breadth-Order Traversal algorithm

```
1 def breadth_order(tnode):  
2     explore = Queue()  
3     explore.enqueue(tnode)  
4  
5     while explore.size() > 0:  
6         current = explore.dequeue()  
7         print(current.get_data(), end=" ")  
8         left = current.get_left()  
9         if left is not None:  
10            explore.enqueue(left)  
11            right = current.get_right()  
12            if right is not None:  
13                explore.enqueue(right)
```

Siblings before children

Calculating tree height

```
1 def height(tnode):  
2     if tnode is None:  
3         return 0  
4     else:  
5         lh = height(tnode.get_left())  
6         rh = height(tnode.get_right())  
7         return 1 + max(lh, rh)
```

Counting nodes in a tree

```
1 def count(tnode):
2     if tnode == None:
3         return 0
4     else:
5         return 1 + count(tnode.get_left()) \
6                     + count(tnode.get_right())
```

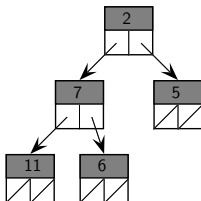
Searching for a data value in a tree

```
1 def member(tnode, val):
2     if tnode == None:
3         return False
4     elif tnode.get_data() == val:
5         return True
6     else:
7         return member(tnode.get_left(), val) \
8             or member(tnode.get_right(), val)
```

Algorithms on trees

- Almost always mirror the formal definition of trees.
- Frequently perform an implied traversal.
- Very easy after a while.

Exercises



Define the following functions:

- To sum the data values in a tree.
- To find the smallest data value in a tree.
- To substitute a target value with a replacement value
- To swap the left and right branches in the tree.
- To find the level of the leaf closest to the root.